CS-1201 Object-Oriented Programming

Inheritance

Arbish Akram

Department of Computer Science
Government College University

The Copy-and-Paste Problem

A university program needs a `Student` class and, later, a `GradStudent` class that is the same plus a thesis title.

```
class Student {
    string name;
    double cgpa;
public:
    void setName(string n) { name = n; }
    string getName() const { return name; }
    void setCgpa(double g)  { cgpa = g; }
    double getCgpa() const { return cgpa; }
};
```

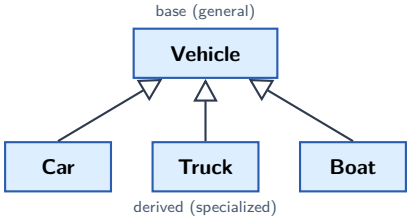
```
class GradStudent {
    string name;              // copied
    double cgpa;              // copied
    string thesis;            // new
public:
    void setName(string n) { name = n; }
    string getName() const { return name; }
    // ... every other line copied too ...
    void setThesis(string t) { thesis = t; }
};
```

- Every copied line is a second place to fix the same bug.
- If `Student` later validates cgpa, someone must remember to change `GradStudent` too.

Inheritance lets a new class **absorb** an existing class's data and functions, then **add** or **customize** features. Common code is written once, in one place.

Base Classes and Derived Classes

- The existing class is the *base class*; the new class is the *derived class*.
- Other names: *superclass* and *subclass* (Java, C#), or *parent* and *child*.
- A derived class is a **more specialized** version of its base class; the base class is **more general**.
- Inheritance models an **is-a** relationship: every `Car` *is a* `Vehicle`, but not every `Vehicle` is a `Car`. It might be a `Truck` or a `Boat`.
- The base class is **not changed** by deriving from it, and it knows nothing about its derived classes.



In UML, inheritance is called **generalization**. The hollow triangle points to the **base** class.

Is It Really Is-a?

Sentence	Sounds right?	Use
A student *is a* person	yes	inheritance
A car *has an* engine	yes	composition
A box *is a* rectangle	doubtful	composition
An employee *is a* date	no	composition

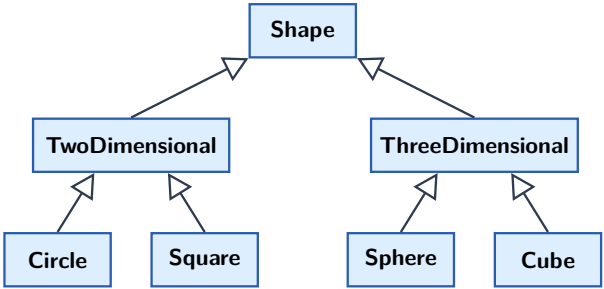
- Say the relationship as a sentence. If "is a" sounds wrong, do not inherit.
- A box *has a* rectangular base; it is not itself a rectangle. An employee *has a* birth date.
- Everything a base object can do, a derived object must be able to do sensibly.

Factor out what classes have in common into a base class, and use inheritance only for true **is-a** relationships. For **has-a**, use composition (previous lecture).

Inheritance Hierarchies

Base class	Derived classes
`Student`	`GradStudent`, `Undergrad`
`Shape`	`Circle`, `Triangle`, `Cube`
`Loan`	`CarLoan`, `HomeLoan`
`Employee`	`Faculty`, `Staff`
`Account`	`Checking`, `Savings`

- A *direct* base class is the one a class derives from immediately (one level up).
- An *indirect* base class is two or more levels up.



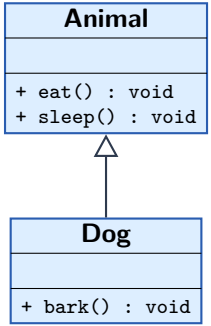
`Shape` is a **direct** base of `TwoDimensional` and an **indirect** base of `Circle`.

Benefits of Inheritance

- **Reuse.** Once a base class is written and debugged, it is reused without being rewritten, which saves time and improves reliability.
- **One place to change.** A fix or improvement in the base class reaches every derived class automatically.
- **The base stays intact.** Deriving a new class never changes the base class's source code.
- **Libraries.** You can derive from a class you only have as a header and compiled object code, without seeing its source.
- **Clearer design.** A hierarchy mirrors how we already classify things: general concepts at the top, specific ones below.

A First Example

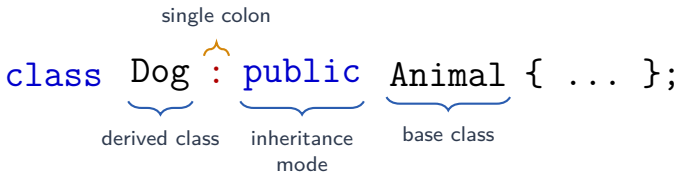
```
1  class Animal {                    // base class
2  public:
3      void eat() const   { cout << "I can eat!" << endl; }
4      void sleep() const { cout << "I can sleep!" << endl; }
5  };
6
7  class Dog : public Animal {       // derived class
8  public:
9      void bark() const {
10         cout << "I can bark! Woof woof!!" << endl;
11     }
12 };
13
14 int main() {
15     Dog dog1;
16     dog1.eat();       // inherited from Animal
17     dog1.sleep();     // inherited from Animal
18     dog1.bark();      // Dog's own member
19     return 0;
20 }
```



Output:

```
I can eat!
I can sleep!
I can bark! Woof woof!!
```

Anatomy of the Derived-Class Header



- The **single** colon means "is derived from". Do not confuse it with the scope resolution operator `::`.
- The base class must be **fully defined** before the derived class; a forward declaration is not enough.
- The derived class only adds what is new. Inherited members are not written again.

Private Members Are Hidden from Derived Classes

```
class Animal {
private:
    string color;
public:
    void setColor(string c) { color = c; }
    string getColor() const { return color; }
};
class Dog : public Animal {
public:
    void describe() const {
        cout << color;          // error
        cout << getColor();     // OK
    }
};
```

```
error: ‘std::string Animal::color’ is private
within this context
```

- `color` is inside every `Dog`, but only `Animal`'s own member functions (and friends) can name it.
- The derived class uses the base's **public interface**, exactly as `main` must.
- Making `color` `public` would fix this, but then *any* code could change it, and data hiding is lost.

C++ offers a level in between: `protected`.

Three Levels of Access

A member declared as ...	its own class	a derived class	outside code (`main`)
`public`	yes	yes	yes
`protected`	yes	yes	no
`private`	yes	no	no

- "Accessible" means the code can use the member **by name**. Friends of a class get the same access as its own members.
- `protected` members behave like `private` ones to the outside world, and like `public` ones to derived classes.

`protected` gives **derived classes** direct access while still hiding the member from everyone else.

Example: private, protected and public

```
class Animal {
private:
    string color;        // Animal only
protected:
    string type;         // Animal + derived
public:
    void eat() const { cout << "I can eat!" << endl; }
    void setColor(string c) { color = c; }
    string getColor() const { return color; }
};
class Dog : public Animal {
public:
    void setType(string t) { type = t; }  // OK
    void displayInfo() const {
        cout << "I am a " << type << endl;
        cout << "My color is " << getColor()
             << endl;
        // cout << color;  // error: private
    }
};
```

```
int main() {
    Dog dog1;
    dog1.setColor("black"); // Animal's
    dog1.setType("mammal"); // Dog's
    dog1.eat();
    dog1.displayInfo();
    // dog1.type = "fish";  // error
    return 0;
}
```

Output:

```
I can eat!
I am a mammal
My color is black
```

`type`: usable in `Dog`, not in `main`. `color`: only via `getColor()`.

The Danger of protected Data

```
class Account {
protected:
    double balance{0};
public:
    void deposit(double amt) {
        if (amt > 0) balance += amt;   // checked
    }
};
class Savings : public Account {
public:
    void oops() { balance = -5000; }  // compiles!
};
```

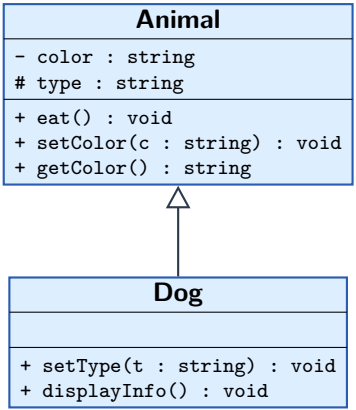
1. Derived code can **skip the validation** in `deposit` and put the object in an invalid state.
2. Derived code depends on the base's **implementation**. Rename `balance` and every derived class breaks: a *fragile* design.

Good practice

- Keep base-class **data** `private`.
- Give derived classes `public` or `protected` **functions** that check every change.
- Use `protected` for helper *functions* meant only for derived classes.

Anyone can derive from your class, so `protected` data is only slightly safer than `public` data. Calling a `get` function costs almost nothing: the compiler usually inlines it.

Drawing Inheritance in UML



- A solid line with a **hollow triangle** pointing to the **base** class.
- Visibility markers: `-` `private`, `#` `protected`, `+` `public`.
- The derived class lists only what it **adds** or **redefines**; inherited members are not repeated.
- Several children may share one triangle when the arrows are merged into a tree.

We now know all three member access levels. Next: what the `public` in `class Dog : public Animal` does to them.

Same Keywords, Two Different Jobs

```
class Animal {
public:          // (1) member access
    void eat() const;
protected:       // (1) member access
    string type;
private:         // (1) member access
    string color;
};
//          (2) inheritance mode
//              vvvvvv
class Dog : public Animal {
public:          // (1) member access
    void bark() const;
};
```

	Member access specifier	**Inheritance mode**
Where	inside a class body, before members	in the derived-class header, before the base name
Controls	who can use the members **declared** in this class	what the **inherited** members become in the derived class

The mode answers one question: once `Dog` has inherited `eat()` and `type`, **who can still reach them through** `Dog`?

Three Ways to Inherit

```
class Dog : public Animal    { ... };   // public inheritance (by far the most common)
class Dog : protected Animal { ... };   // protected inheritance (rare)
class Dog : private Animal   { ... };   // private inheritance
class Dog : Animal           { ... };   // no keyword: private for a class
```

- The inheritance mode decides what the inherited members become **inside the derived class**, and therefore who outside can still reach them through a derived object.
- It can only make access **tighter**, never looser.
- In **every** mode, the base's `private` members remain hidden from the derived class. They are inherited, but never directly accessible.

Only **public** inheritance means **is-a**. With `private` or `protected` inheritance, outside code cannot treat a `Dog` as an `Animal`.

What Base Members Become in the Derived Class

Base member is ...	`public` **inheritance**	`protected` **inheritance**	`private` **inheritance**
`public`	`public`	`protected`	`private`
`protected`	`protected`	`protected`	`private`
`private`	not accessible	not accessible	not accessible

- "Not accessible" means derived code must use the base's `public` or `protected` functions.
- Derived member functions can always use inherited `public` and `protected` members. The mode only affects code **outside** the derived class and classes derived from it.

Access Combinations in Code

```
class A {
private:   int privA;
protected: int protA;
public:    int pubA;
};
class B : public A {
public:
    void f() {
        int a = privA;   // error: private in A
        int b = protA;   // OK
        int c = pubA;    // OK
    }
};
class C : private A {
public:
    void f() {
        int a = privA;   // error: private in A
        int b = protA;   // OK (now private in C)
        int c = pubA;    // OK (now private in C)
    }
};
```

```
int main() {
    B objB;
    int x = objB.protA;  // error
    int y = objB.pubA;   // OK: public
    C objC;
    int z = objC.pubA;   // error: private
    return 0;
}
```

Inside `B` and `C` the code is identical and behaves the same. The difference appears only in `main`: `pubA` is public through `B` but private through `C`.

protected vs. private Inheritance: One Level Down

```
class A {
public:
    void hello() const { cout << "hello\n"; }
};
class B1 : protected A { }; // hello() protected
class B2 : private A   { }; // hello() private

class C1 : public B1 {
public:
    void f() const { hello(); }  // OK
};
class C2 : public B2 {
public:
    void f() const { hello(); }  // error
};
int main() {
    B1 b1;  b1.hello();   // error: protected in B1
    B2 b2;  b2.hello();   // error: private in B2
}
```

- From `main`, `B1` and `B2` look the same: `hello()` is hidden in both.
- The difference appears in the **next** derived class: `C1` may still call `hello()`, `C2` may not.

Who can still use `A`'s public interface?

`public`: everyone.
`protected`: the derived class and its descendants.
`private`: the derived class only.

When to Use Which Mode

Mode	Use it when ...	Real example	Base functions are usable by ...
`public`	the derived class **is a** kind of the base, and should offer everything the base offers	A savings account *is an* account: it needs `deposit()` too	everyone
`private`	the derived class is only **built using** the base, and must hide the base's functions	A stack *is built using* a list, but must block inserting in the middle	the derived class only
`protected`	as for `private`, but **further derived classes** still need the base's functions	A hospital queue adds emergency patients at the front of a queue	the derived class and its descendants

If "X *is a* Y" is true, use `public`: the right choice almost every time. The next slides show each example in code.

public Inheritance: A Savings Account

```
1  class Account {
2      double balance{0};
3  public:
4      void deposit(double d) { if (d > 0) balance += d; }
5      double getBalance() const { return balance; }
6  };
7  class SavingsAccount : public Account {
8      double rate;
9  public:
10     SavingsAccount(double r) : rate{r} { }
11     void addInterest() { deposit(getBalance() * rate); }
12 };
13 void printBalance(const Account& a) {   // any Account
14     cout << "Balance: " << a.getBalance() << endl;
15 }
16 int main() {
17     SavingsAccount s{0.10};
18     s.deposit(1000);     // OK: deposit() stays public
19     s.addInterest();     // SavingsAccount's own
20     printBalance(s);     // prints  Balance: 1100
21     return 0;
22 }
```

- A savings account **is an** account: customers still deposit money and check the balance.
- `public` keeps `deposit()` public, so `main` can call it on `s`.
- `printBalance` expects an `Account` and accepts `s` too.

With `private`, the calls to `deposit` and `printBalance` in `main` would both fail.

`private` Inheritance: A Stack Built on a List

```
1  class List {                        // array-based list
2      int items[100];
3      int count{0};
4  public:
5      void insertAt(int pos, int v);  // add anywhere
6      int removeAt(int pos);          // remove anywhere
7      int size() const { return count; }
8  };
9  class Stack : private List {
10 public:
11     void push(int v) { insertAt(size(), v); }     // OK
12     int pop() { return removeAt(size() - 1); }    // OK
13 };
14 int main() {
15     Stack s;
16     s.push(10);
17     s.push(20);
18     cout << s.pop() << endl;    // 20
19     // s.insertAt(0, 99);       // error: inaccessible
20     return 0;
21 }
```

- A stack may only add and remove at the **top**. `insertAt(0, 99)` would break that rule.
- `Stack` reuses `List`'s code, while `private` hides its functions from `main`.

A stack is **not** a list, so composition is usually clearer: make `List items;` a member of `Stack`. It hides `List` just the same.

protected Inheritance: A Hospital Queue

```
// List is the same class as on the previous slide
class Queue : protected List {
public:
    void enqueue(int v) { insertAt(size(), v); } // back
    int dequeue() { return removeAt(0); }        // front
};
class HospitalQueue : public Queue {
public:
    void addEmergency(int token) {
        insertAt(0, token);   // OK: protected in Queue
    }
};
int main() {
    HospitalQueue q;
    q.enqueue(101);           // normal patients
    q.enqueue(102);
    q.addEmergency(900);      // goes to the front
    cout << q.dequeue() << endl;   // 900
    cout << q.dequeue() << endl;   // 101
    // q.insertAt(1, 500);    // error: inaccessible
    return 0;
}
```

- Users of a queue must not insert anywhere they like, so `main` cannot call `insertAt()`.
- `HospitalQueue` really needs it: emergency patients go first.
- `protected` allows both.

With `private List`, `addEmergency` would not compile: `insertAt()` would be private in `Queue`. This need is rare; most classes use `public`.

Constructors Are Not Inherited

```
class Counter {
protected:
    unsigned int count;
public:
    Counter() : count{0} { }
    Counter(unsigned int c) : count{c} { }
    unsigned int get_count() const { return count; }
    Counter& operator++() { ++count; return *this; }
};
class CountDn : public Counter {
public:
    CountDn& operator--() { --count; return *this; }
};

CountDn c1;          // OK: Counter() is called
CountDn c2(100);     // error: no CountDn(int)
```

- `CountDn` inherits `get_count()` and `operator++`, and adds `operator--`.
- The compiler gives `CountDn` a default constructor that calls `Counter()`, so `c1` works.
- But `Counter(unsigned int)` is **not** inherited, so `c2(100)` does not compile.
- `count` is `protected` here only to keep the example short.

Fix: give `CountDn` its own constructors that **pass arguments up** to `Counter`.

Calling the Base-Class Constructor

```
1  class CountDn : public Counter {
2  public:
3      CountDn() : Counter() { }                // no args
4      CountDn(unsigned int c) : Counter(c) { } // pass c up
5      CountDn& operator--() { --count; return *this; }
6  };
7
8  int main() {
9      CountDn c1;           // count = 0
10     CountDn c2(100);      // count = 100
11     ++c1; ++c1; ++c1;     // inherited operator++
12     --c2; --c2;           // CountDn's own operator--
13     cout << "c1=" << c1.get_count() << endl;
14     cout << "c2=" << c2.get_count() << endl;
15     return 0;
16 }
```

Output:

```
c1=3
c2=98
```

- The base constructor is called in the *member initializer list*, by the **class name**: `: Counter(c)`.
- It runs **before** the derived constructor's body, so the inherited part is ready before the derived class uses it.
- The base constructor initializes the base's data, including its `private` members, which the derived constructor could not touch anyway.

Order of Constructor and Destructor Calls

```
1  class Part {
2      string name;
3  public:
4      Part(string n) : name{n}
5          { cout << "  Part " << name << " built\n"; }
6      ~Part()
7          { cout << "  Part " << name << " destroyed\n"; }
8  };
9  class Base {
10     Part p{"of Base"};              // member object
11 public:
12     Base()  { cout << "Base constructor\n"; }
13     ~Base() { cout << "Base destructor\n"; }
14 };
15 class Derived : public Base {
16     Part q{"of Derived"};           // member object
17 public:
18     Derived()  { cout << "Derived constructor\n"; }
19     ~Derived() { cout << "Derived destructor\n"; }
20 };
21 int main() { Derived d; cout << "--- end ---\n"; }
```

Output:

```
  Part of Base built
Base constructor
Part of Derived built
Derived constructor
-- end --
Derived destructor
Part of Derived destroyed
Base destructor
Part of Base destroyed
```

Base before derived; members before the body. Destruction is the exact **reverse**.

Redefining a Member Function

```
class Stack {
protected:
    static const int MAX = 3;
    int st[MAX];
    int top{-1};                // index of top item
public:
    void push(int v) { st[++top] = v; }  // no checks
    int pop()        { return st[top--]; }
};
class Stack2 : public Stack {   // a safer stack
public:
    void push(int v) {          // same name and params
        if (top >= MAX - 1) {
            cout << "Error: stack is full\n";
            return;
        }
        Stack::push(v);         // reuse base version
    }
};
```

A matching `Stack2::pop()` would check for an empty stack the same way, then call `Stack::pop()`.

```
int main() {
    Stack2 s;
    s.push(11); s.push(22); s.push(33);
    s.push(44);              // too many
    cout << s.pop() << endl;
    cout << s.pop() << endl;
    cout << s.pop() << endl;
    return 0;
}
```

```
Error: stack is full
33
22
11
```

`Stack2` keeps `Stack`'s storage and logic, and only adds the safety check. `pop()` is inherited unchanged.

Which Function Runs?

- When the **same function** (same name and parameters) exists in the base and the derived class, a call on a **derived** object runs the **derived** version. We say the derived function *redefines* (or overrides) the base one.
- Base objects know nothing about derived classes and always use the base version.
- Inside the derived class, the base version is still reachable with the **scope resolution operator**: `Stack::push(v)`.

Classic bug: forgetting `Stack::`

```
void push(int v) {
    if (top >= MAX - 1) return;
    push(v);    // calls itself:
}               // infinite recursion
```

Overloading vs. redefining

Overloading: same name, *different* parameters, usually in the same class.

Redefining: same name, *same* parameters, in a derived class.

A Trap: Name Hiding

```
class Printer {
public:
    void print(int n) const
        { cout << "int: " << n << endl; }
    void print(string s) const
        { cout << "text: " << s << endl; }
};
class ColorPrinter : public Printer {
public:
    void print(string s) const      // redefines ONE
        { cout << "[color] " << s << endl; }
};
int main() {
    ColorPrinter cp;
    cp.print("hello");   // OK: [color] hello
    cp.print(5);         // error!
    return 0;
}
```

```
error: cannot convert ‘int’ to ‘std::string’
```

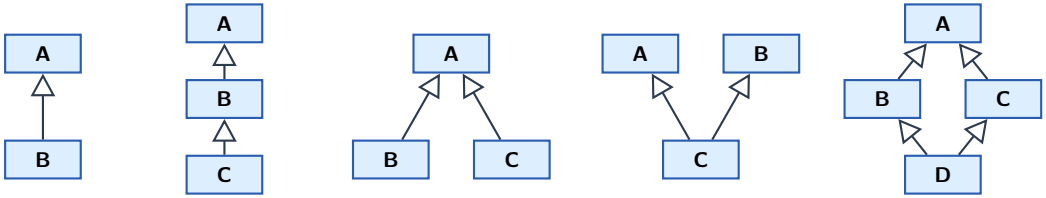
- Declaring **any** `print` in `ColorPrinter` **hides every** `print` of `Printer`, whatever its parameters.
- The compiler finds `print` in `ColorPrinter`, stops looking, and fails to convert 5 to a `string`.

Fix: bring the base versions back into scope.

```
class ColorPrinter : public Printer {
public:
    using Printer::print;
    void print(string s) const;
};
```

Now `cp.print(5)` prints `int: 5`.

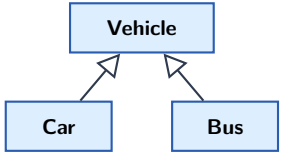
Forms of Inheritance



Single	one derived class, one base class
Multilevel	a chain: `C` derives from `B`, which derives from `A`
Hierarchical	several classes derive from the same base
Multiple	one class derives from two or more bases at once
Hybrid	a mix of the above; the shape shown is the “diamond”

Hierarchical Inheritance: Car and Bus

```cpp
class Vehicle {
    string brand;
    int year;
public:
    Vehicle(string b, int y) : brand{b}, year{y} { }
    void displayInfo() const {
        cout << brand << " (" << year << ")";
    }
};
class Car : public Vehicle {
    int doors;
public:
    Car(string b, int y, int d)
        : Vehicle(b, y), doors{d} { }
    void display() const {
        displayInfo();
        cout << ", " << doors << " doors" << endl;
    }
};
// class Bus : public Vehicle  is the same,
// with  int seats  instead of  int doors
```




```cpp
Car car{"Toyota", 2020, 4};
Bus bus{"Hino", 2018, 52};
car.display();
bus.display();
```

```
Toyota (2020), 4 doors
Hino (2018), 52 seats
```

The code for brand and year is written **once** and shared by both derived classes.

Multilevel Inheritance: A Chain of Classes

```cpp
class Person {
    string name;
public:
    Person(string n) : name{n} { }
    void display() const {
        cout << "Name: " << name << endl;
    }
};
class Employee : public Person {
    string company;
public:
    Employee(string n, string c)
        : Person(n), company{c} { }
    void display() const {
        Person::display();
        cout << "Company: " << company
             << endl;
    }
};
```

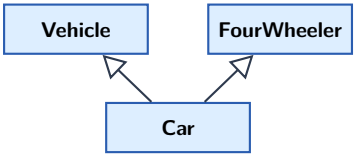
Each class calls only its **direct** base, in the constructor and in `display()`.

```cpp
class Programmer : public Employee {
    int langs;
public:
    Programmer(string n, string c,
               int l)
        : Employee(n, c), langs{l} { }
    void display() const {
        Employee::display();
        cout << "Languages: " << langs
             << endl;
    }
};
int main() {
    Programmer p{"Bilal", "Systems", 4};
    p.display();
}
```

```
Name: Bilal
Company: Systems
Languages: 4
```

Multiple Inheritance

```
class Vehicle {
public:
    Vehicle() { cout << "This is a Vehicle\n"; }
    void display() const
        { cout << "disp of Vehicle\n"; }
};
class FourWheeler {
public:
    FourWheeler() { cout << "This is a 4 wheeler\n"; }
    void display() const
        { cout << "disp of FourWheeler\n"; }
};
// one class, two base classes
class Car : public Vehicle, public FourWheeler {
public:
    Car() { cout << "This is a car\n"; }
};
```



- Bases are separated by commas, and **each** needs its own mode.
- Base constructors run in the order the bases are **listed in the class header**, not the order in the initializer list.

`Car obj;` prints:

```
This is a Vehicle
This is a 4 wheeler
This is a car
```

Ambiguity: The Same Name in Two Bases

```
int main() {
    Car obj;
    obj.display();               // error
    obj.Vehicle::display();      // OK
    obj.FourWheeler::display(); // OK
    return 0;
}
```

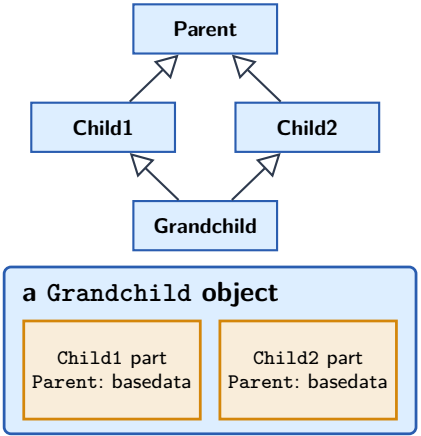
```
error: request for member ‘display’ is
ambiguous
```

- Both bases offer `display()`, and `Car` does not, so the compiler cannot choose.
- Naming the class with `::` removes the ambiguity.

Who defines `display()`?	`obj.display()`
`Car` itself	`Car`’s version
only one of the bases	that base’s version
both bases, not `Car`	compile error

A clean fix: redefine `display()` in `Car` and call the base versions you want inside it.

The Diamond Problem




```
1  class Parent {
2  protected:
3      int basedata{7};
4  };
5  class Child1 : public Parent { };
6  class Child2 : public Parent { };
7  class Grandchild : public Child1, public Child2 {
8  public:
9      int getdata() const {
10         return basedata;    // error: ambiguous
11     }
12 };
```

- `Child1` and `Child2` each contain their own `Parent` *subobject*.
- So a `Grandchild` holds **two** copies of `basedata`, and the compiler cannot tell which one is meant.

Virtual Base Classes

```
class Parent {
protected:
    int basedata{7};
};
class Child1 : virtual public Parent { };  // share
class Child2 : virtual public Parent { };  // share
class Grandchild : public Child1, public Child2 {
public:
    int getdata() const {
        return basedata;    // OK: only one copy
    }
};
int main() {
    Grandchild g;
    cout << g.getdata() << endl;   // 7
    return 0;
}
```

- The keyword `virtual` in the **middle** classes tells them to **share** one `Parent` subobject.
- `Grandchild` now holds a single `basedata`, and the ambiguity is gone.
- `virtual` goes on `Child1` and `Child2`, not on `Grandchild`.

Needing a virtual base class is often a sign that the hierarchy can be designed more simply. Use multiple inheritance with care.

Summary

1. **Why:** a **derived** class absorbs a **base** class and adds or customizes features. Inherit only for **is-a**; use composition for **has-a**.
2. **How:** `class D : public B`. All data is inherited, even `private`; constructors, the destructor, `operator=` and friends are **not**.
3. **Access:** `private` members are for the base only, `protected` ones for the base and derived classes. Prefer `private` data with `protected`/`public` functions.
4. **Modes:** an inherited member gets the **stricter** of its own access and the mode. Base `private` members are never accessible. Only `public` inheritance is is-a.
5. **Building:** call the base constructor in the initializer list. Order: base, members, derived body; destruction in reverse.
6. **Customizing:** a same-signature function **redefines** the base one; reach it with `Base::f()`. A same-named function hides *all* base overloads (`using` fixes it).
7. **Shapes:** single, multilevel, hierarchical, multiple, hybrid. Multiple inheritance may need `::` for ambiguity and `virtual` bases for the diamond.